# COURSEWORK 1: STRUCTURAL INSPECTION PATH PLANNING

COMP0240: AERIAL ROBOTICS: FUNDAMENTALS TO PRACTICE IN REAL-WORLD ENVIRONMENTS

**Amir Gerges**
University College London
Word Count: 1381

March 11, 2026

## 1 Problem Statement and Objectives

For autonomous drone-based structural inspection, the drone must visit all viewpoints to capture images of ArUco markers while avoiding static cuboid obstacles in a three-dimensional environment. The challenge decomposes into two coupled sub-problems: determining the optimal order in which viewpoints should be visited and computing collision-free trajectories between consecutive viewpoints. This paper is focused on the coverage path planning problem within the Gazebo simulator environment using ROS2 and Aerostack2. The objectives are to implement and compare three TSP solvers (Greedy nearest-neighbour, Held-Karp dynamic programming, and NetworkX approximation) and four motion planners (A*, Dijkstra, RRT, and RRT*) across four scenarios of varying complexity, producing 48 test runs to identify the configuration that best balances computational speed, path quality, and robustness. The algorithms fall into two categories: graph-search and sampling-based motion planning. Both categories have their own variants for optimality.

## 2 Methodology

### 2.1 TSP Formulation

Based on the $n$ viewpoints and the starting point, the Euclidean distance matrix $D$ can be obtained, where the entry $D_{ij}$ represents the straight line distance between the nodes (Equation 1).

$$D_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2} \tag{1}$$

There are three TSP solvers that can be applied based on the distance matrix $D$. The Greedy nearest neighbour heuristic algorithm [1] can be used, which chooses the nearest viewpoint from the current viewpoint (Equation 2) with a time complexity of $O(n^2)$.

$$v_{\text{next}} = \arg\min_{j \in U} D(v_{\text{current}}, j) \tag{2}$$

where $U$ is the set of unvisited viewpoints. The Held–Karp algorithm [2] based on dynamic programming can be applied for the optimal solution with a time complexity of $O(n^2 \cdot 2^n)$ by using the python_tsp library, which is based on the Held–Karp bitmask recurrence algorithm. The NetworkX approximation [3] uses the `nx.approximation.greedy_tsp` function, a polynomial-time heuristic without optimality guarantees.

### 2.2 Path Planning Algorithms

Between consecutive viewpoints, a collision-free path is computed through the obstacle field on a 3D occupancy grid (0.5 m resolution, 26-connected neighbourhood). A* [4] expands nodes using the evaluation function in Equation 3, where $g(n)$ is the accumulated cost from the start and $h(n)$ is the 3D Euclidean heuristic. Because the Euclidean distance never overestimates the true grid path cost, A* is guaranteed to return the shortest grid path.

$$f(n) = g(n) + h(n) \tag{3}$$

Dijkstra's algorithm [5] expands nodes purely by accumulated cost, using the same 3D grid and 26-connected neighbourhood as A* but without any heuristic guidance. Where A* prioritises nodes by $f = g + h$, Dijkstra prioritises by $g$ alone, meaning it explores all directions uniformly until reaching the goal. This serves as a controlled baseline to measure the benefit of A*'s heuristic.

RRT [6] builds a search tree by randomly sampling the space with 10% goal bias. At each iteration, the nearest existing node is identified and a new node is steered towards the sample using a fixed step size (Equation 4).

$$\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{near}} + \min(s, \|\mathbf{x}_{\text{rand}} - \mathbf{x}_{\text{near}}\|) \cdot \frac{\mathbf{x}_{\text{rand}} - \mathbf{x}_{\text{near}}}{\|\mathbf{x}_{\text{rand}} - \mathbf{x}_{\text{near}}\|} \tag{4}$$

where $s = 0.5$ m is the step size and the tree is grown for up to 5000 iterations. RRT* [7] extends RRT with eventual optimality by rewiring the tree whenever shorter connections are discovered. The implementation uses the OMPL library with a 10.0 s planning budget per segment.

### 2.3 System Integration

Obstacles are modeled as axis-aligned bounding boxes expanded by 1.0 m along each axis to ensure collision safety. The grid spans $\pm 20$ m in the X and Y axes and 0 to 15 m in the Z axis, totalling $80 \times 80 \times 30$ grid cells. For collision detection, 21 points are sampled per segment. A line of sight step removes unnecessary waypoints by greedily skipping ahead. Waypoints are published as `nav_msgs/Path` and executed via Aerostack2's `follow_path` at 1.0 m/s with fixed yaw towards the marker. ArUco detection using OpenCV's DICT_5X5_250 dictionary verifies markers with a 3.0 s timeout. The distance overhead is defined as the percentage increase of actual flight distance over the TSP straight-line distance.

## 3 Results and Discussion

### 3.1 Experimental Setup

All 48 combinations of the three TSP methods and the four planners were executed for the four scenarios. All simulations resulted in 100% mission success, with all ArUco markers verified, therefore ensuring the reliability of the collision avoidance system. As the TSP results are independent of the planner, A* was fixed for the TSP evaluation. As discussed in Section 3.2, the planner evaluation was conducted using the Greedy TSP. A table containing all the results is presented in Appendix A.

### 3.2 TSP Solver Comparison

TSP computation times (Figure 1) reveal a scalability contrast. Greedy completes in 0.1–0.5 ms across all scenarios, while DP surges from 21.1–27.5 ms at 10 viewpoints (S1/S2) to 160.91 s at 20 viewpoints (S3) roughly 322,000$\times$ slower than Greedy. NetworkX remains fast at 0.2–1 ms, comparable to Greedy. Path quality (Figure 2) shows DP produces the shortest tours at S3, DP achieves 80.4 m against 99.8 m for both Greedy and NetworkX, saving 19.4%. At S1/S2 all three produce near identical distances of 64.58–67.15 m, and at S4 all converge to 51.81 m as five viewpoints leave minimal ordering variation. NetworkX matches Greedy's tours across every scenario with no speed advantage. The 19.4 m saving at S3 does not justify DP's 160.91 s computation, which exceeds the Greedy + A* mission time of 154.6 s (Figure 5). This identifies Greedy as the practical TSP choice.

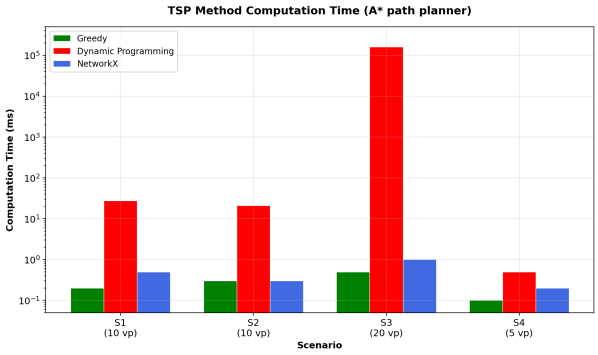



Figure 1: TSP computation time across scenarios (A* planner)

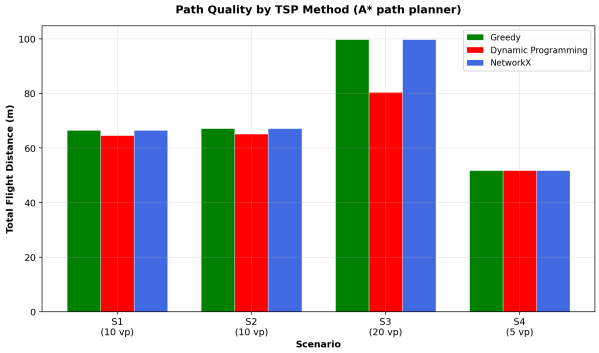



Figure 2: TSP distance across scenarios (A* planner)

### 3.3 Path Planner Comparison

With Greedy TSP held constant, the planner comparison (Figure 3) shows A* computes all segment paths in 170.4–303.2 ms total, while Dijkstra requires 11.94–15.77 s approximately 47–71× slower. Dijkstra expands every reachable node up to the goal's cost contour, whereas A*'s Euclidean heuristic (Equation 3) biases expansion towards the goal, pruning the $80 \times 80 \times 30$ search space. The distance overhead (Figure 4) confirms this speed gain incurs no path quality loss: A* achieves 2.5% average / 8.4% maximum overhead and Dijkstra 2.7% / 8.6% near identical, as both find optimal grid paths.

RRT exhibits the highest overhead, averaging 5.8% and peaking at 18.0% in S4, where dense obstacles cause extensive random exploration. RRT* reduces this to 2.7% average / 10.0% maximum through rewiring, but at 50.18–200.70 s computation. Both sampling-based planners also introduce run-to-run variance undesirable for mission critical inspections. This identifies A* as the practical planner choice: 47–71× faster than Dijkstra with identical path quality, lower overhead than RRT/RRT*, and fully deterministic. More broadly, the structured and bounded 3D grid inherently favours graph-search methods, which exploit the finite discretised space, over sampling-based planners that expend iterations exploring free space without such structure.

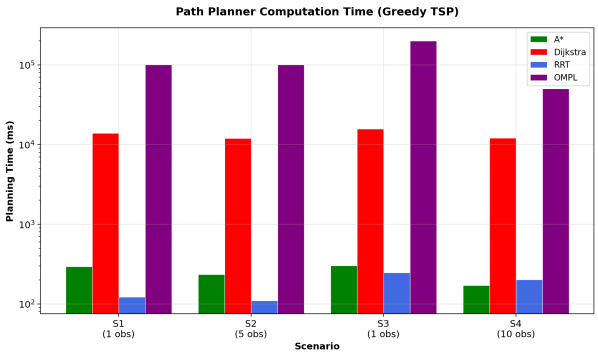



Figure 3: Planner computation time across scenarios (Greedy TSP)

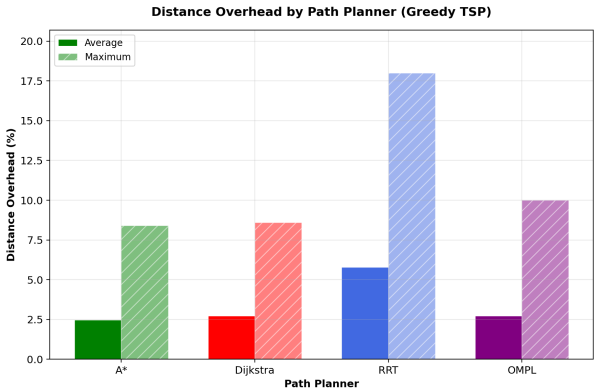



Figure 4: Distance overhead across scenarios (Greedy TSP)

### 3.4 Mission Time and Trajectory Analysis

Mission time (Figure 5) combines planning and flight. A* completes in 73.29–154.6 s, fastest overall. Dijkstra adds 11.20–12.37 s with no distance gain. RRT shows 144.33 s (S3) and 70.83 s (S4). RRT* requires 120.23–343.92 s from its 10.0 s per-segment budget. The trajectories (Figures 6–9) confirm the flight path follows the TSP tour in low obstacle scenarios but deviates in S4's dense obstacle field.

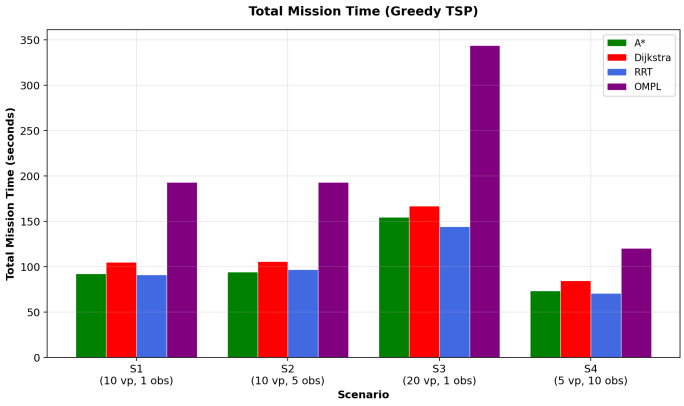



Figure 5: Total mission time across scenarios (Greedy TSP)

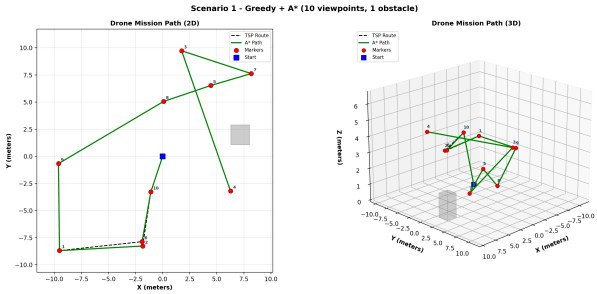



Figure 6: Trajectory S1 (10 viewpoints, 1 obstacle)

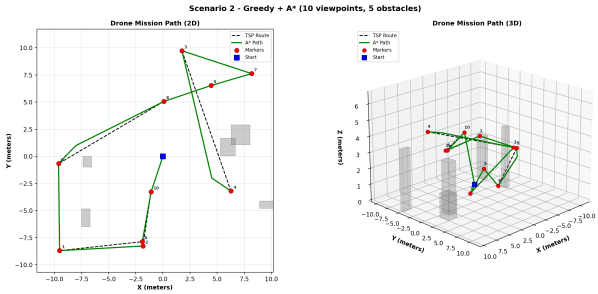



Figure 7: Trajectory S2 (10 viewpoints, 5 obstacles)

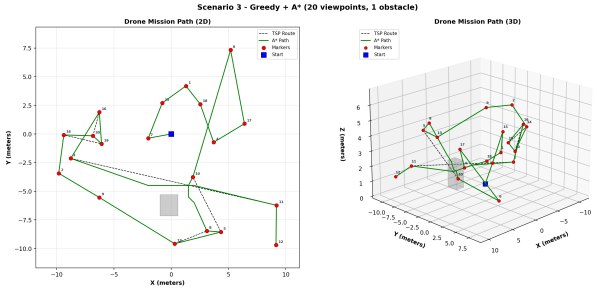



Figure 8: Trajectory S3 (20 viewpoints, 1 obstacle)

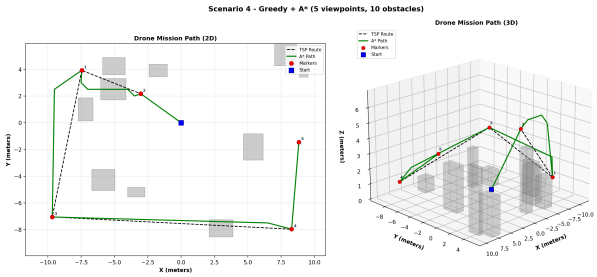



Figure 9: Trajectory S4 (5 viewpoints, 10 obstacles)

### 3.5 Limitations

Several assumptions limit generality. The TSP uses Euclidean distances rather than obstacle aware path lengths, meaning the optimal Euclidean distances may not correspond to the optimal flight distance when obstacles force detours as shown in S4 where 8.4% overhead arises despite identical TSP tours. The environment is fully static so dynamic obstacles would require replanning. The 0.5 m grid resolution introduces path rounding effects, and the constant 1.0 m/s speed ignores acceleration dynamics and energy constraints.

## 4 Conclusion

The performance of twelve combinations of TSP Planner configurations over four scenarios is evaluated within this work, resulting in a total of 48 test runs with complete mission success. The Greedy + A* configuration is the recommended choice: Greedy delivers sub-millisecond tour computation with tours within 19.4 m of the DP optimum at S3, while A* provides the fastest planning (170.4–303.2 ms) with the lowest overhead (2.5% average, 8.4% maximum)

and fully deterministic behaviour. The key result of this work is that the heuristic guidance provided by the nearest neighbour ordering and the Euclidean heuristic $h(n)$ for pathfinding provides the largest performance improvements, while exact algorithms show diminishing returns with exponentially increasing costs. A notable finding is that RRT*'s theoretical optimality guarantee did not translate into practical advantage within this bounded discrete grid, suggesting that environment structure should guide planner selection. Future work should aim to derive the TSP distance matrix from path lengths, implement dynamic replanning for moving obstacles, and develop energy optimisation for trajectories with battery and acceleration limitations.

## References

[1] D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis, "An Analysis of Several Heuristics for the Traveling Salesman Problem," *SIAM Journal on Computing*, vol. 6, no. 3, pp. 563–581, 1977.

[2] M. Held and R. M. Karp, "A Dynamic Programming Approach to Sequencing Problems," *Journal of SIAM*, vol. 10, no. 1, pp. 196–210, 1962.

[3] A. Hagberg, D. Schult, and P. Swart, "Exploring Network Structure, Dynamics, and Function using NetworkX," in *Proc. 7th Python in Science Conference*, pp. 11–15, 2008.

[4] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[5] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[6] S. M. LaValle, "Rapidly-Exploring Random Trees: A New Tool for Path Planning," Technical Report, Iowa State University, 1998.

[7] S. Karaman and E. Frazzoli, "Sampling-based Algorithms for Optimal Motion Planning," *International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.

# Appendix A

## Complete Experimental Results

| Scenario | TSP | Planner | Dist (m) | Overhead % | Time (s) | TSP (ms) | Plan (ms) |
|---|---|---|---|---|---|---|---|
| S1 | DP | A* | 64.58 | 1.5 | 92.38 | 27.5 | 158.6 |
| S1 | DP | Dijkstra | 64.58 | 1.5 | 102.29 | 25.8 | 10900.1 |
| S1 | DP | RRT | 64.94 | 2.1 | 91.27 | 22.1 | 112.6 |
| S1 | DP | OMPL | 64.72 | 1.7 | 193.82 | 23.0 | 100865.3 |
| S1 | Greedy | A* | 66.55 | 0.0 | 92.38 | 0.2 | 295.2 |
| S1 | Greedy | Dijkstra | 66.55 | 0.0 | 104.75 | 0.2 | 13853.6 |
| S1 | Greedy | RRT | 66.55 | 0.0 | 91.19 | 0.3 | 122.0 |
| S1 | Greedy | OMPL | 66.55 | 0.0 | 192.83 | 0.2 | 100660.0 |
| S1 | NX | A* | 66.55 | 0.0 | 91.19 | 0.5 | 297.4 |
| S1 | NX | Dijkstra | 66.55 | 0.0 | 103.08 | 0.4 | 12109.6 |
| S1 | NX | RRT | 67.82 | 1.9 | 93.82 | 0.4 | 149.0 |
| S1 | NX | OMPL | 66.55 | 0.0 | 191.87 | 0.4 | 100623.1 |
| S2 | DP | A* | 65.13 | 2.4 | 99.59 | 21.1 | 113.6 |
| S2 | DP | Dijkstra | 65.39 | 2.8 | 108.24 | 26.1 | 9349.9 |
| S2 | DP | RRT | 75.72 | 19.0 | 100.14 | 22.3 | 932.5 |
| S2 | DP | OMPL | 64.86 | 1.9 | 194.04 | 26.5 | 100681.2 |
| S2 | Greedy | A* | 67.15 | 0.9 | 94.01 | 0.3 | 233.0 |
| S2 | Greedy | Dijkstra | 67.15 | 0.9 | 105.82 | 0.2 | 11944.1 |
| S2 | Greedy | RRT | 69.88 | 5.0 | 96.79 | 0.2 | 109.6 |
| S2 | Greedy | OMPL | 67.15 | 0.9 | 193.16 | 0.2 | 100899.6 |
| S2 | NX | A* | 67.15 | 0.9 | 94.02 | 0.3 | 242.5 |
| S2 | NX | Dijkstra | 67.15 | 0.9 | 105.66 | 0.4 | 11882.4 |
| S2 | NX | RRT | 69.59 | 4.6 | 95.58 | 0.4 | 190.8 |
| S2 | NX | OMPL | 67.10 | 0.8 | 193.69 | 1.4 | 100831.3 |
| S3 | DP | A* | 80.40 | 0.0 | 123.62 | 160914.8 | 181.1 |
| S3 | DP | Dijkstra | 80.40 | 0.0 | 130.69 | 161183.0 | 6191.7 |
| S3 | DP | RRT | 80.40 | 0.0 | 125.13 | 159225.5 | 136.9 |
| S3 | DP | OMPL | 80.40 | 0.0 | 324.80 | 161226.6 | 201222.7 |
| S3 | Greedy | A* | 99.80 | 0.6 | 154.60 | 0.5 | 303.2 |
| S3 | Greedy | Dijkstra | 100.63 | 1.4 | 166.78 | 0.6 | 15772.4 |
| S3 | Greedy | RRT | 99.37 | 0.2 | 144.33 | 0.9 | 246.6 |
| S3 | Greedy | OMPL | 99.21 | 0.0 | 343.92 | 0.7 | 200696.1 |
| S3 | NX | A* | 99.80 | 0.6 | 154.94 | 1.0 | 321.4 |
| S3 | NX | Dijkstra | 100.63 | 1.4 | 167.19 | 1.6 | 15690.5 |
| S3 | NX | RRT | 101.29 | 2.1 | 144.85 | 1.0 | 173.3 |
| S3 | NX | OMPL | 99.22 | 0.0 | 345.47 | 0.9 | 200868.3 |
| S4 | DP | A* | 51.81 | 8.4 | 74.12 | 0.5 | 184.3 |
| S4 | DP | Dijkstra | 51.88 | 8.6 | 85.02 | 0.5 | 12090.5 |
| S4 | DP | RRT | 58.04 | 21.5 | 87.82 | 0.3 | 13584.7 |
| S4 | DP | OMPL | 53.71 | 12.4 | 120.91 | 0.4 | 50254.4 |
| S4 | Greedy | A* | 51.81 | 8.4 | 73.29 | 0.1 | 170.4 |
| S4 | Greedy | Dijkstra | 51.88 | 8.6 | 84.49 | 0.1 | 12062.0 |
| S4 | Greedy | RRT | 56.39 | 18.0 | 70.83 | 0.1 | 201.2 |
| S4 | Greedy | OMPL | 52.56 | 10.0 | 120.23 | 0.1 | 50178.2 |
| S4 | NX | A* | 51.81 | 8.4 | 74.09 | 0.2 | 168.6 |
| S4 | NX | Dijkstra | 51.88 | 8.6 | 84.86 | 0.2 | 12022.3 |
| S4 | NX | RRT | 59.02 | 23.5 | 84.71 | 0.2 | 11370.7 |
| S4 | NX | OMPL | 52.39 | 9.6 | 118.58 | 0.2 | 50232.1 |

Table 1: All 48 experimental runs. NX = NetworkX, OMPL = RRT*